

Software Requirements Specification (SRS) Document for Trakify

Table of Contents

Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

1.5 Overview

Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 User Documentation

2.7 Assumptions and Dependencies

External Interface Requirements

3.1 User Interfaces

3.2 Hardware Interfaces

3.3 Software Interfaces

3.4 Communication Interfaces

System Features

4.1 Add Habit

4.2 Daily Tracking

4.3 Statistics

Non-functional Requirements

5.1 Performance Requirements

5.2 Security Requirements

5.3 Usability Requirements

5.4 Reliability and Availability

6. Other Requirements

7. User Stories

8. Use Case

9. UML & Diagrams

10. ERD – Entity Relationship Diagram

11. Wireframes – Initial Screen Sketches

12. Technologies Used

1. Introduction

1.1 Purpose

Trakify is a daily habit-tracking application aimed at helping users develop a healthy and productive routine. It features an interactive interface that allows users to create habits, track progress, and receive reminder notifications.

1.2 Scope

The app provides various functionalities, including creating new habits, setting duration and specific days, displaying habits in a daily list format, and monitoring daily and weekly statistics. Trakify is designed to be user-friendly and targets young people and adults looking to improve their lifestyle through a smart digital solution.

1.3 Definitions, Acronyms, and Abbreviations

UI: User Interface

UX: User Experience

SRS: Software Requirements Specification

JWT: JSON Web Token

Habit: A behavior repeated regularly by the user (e.g., drinking water or exercising)

1.4 References

Figma Design File: [Figma](#)

Flutter Documentation: [Flutter](#)

Node.js Documentation: [Node.js](#)

Express Documentation: [Express](#)

MongoDB Documentation: [MongoDB](#)

1.5 Overview

This document outlines all the requirements for the Trakify app, from interface descriptions to functional and non-functional specifications.

2. Overall Description

2.1 Product Perspective

Trakify is a standalone habit-tracking app, developed for Android and iOS platforms using Flutter. It does not rely on external systems but integrates with the device's native notification system.

2.2 Product Functions

Record daily habits

Customize habits by time and days

Reminder notifications

Display daily achievement status

Provide weekly reports

2.3 User Classes and Characteristics

Regular User: Creates and tracks habits

Beginner User: Needs a simple and easy-to-use interface

Advanced User: Interested in detailed statistics and reports

2.4 Operating Environment

Flutter SDK

Android 8+ / iOS 12+

Backend: Node.js / Express

Database: MongoDB

Local Storage and Cache

Cloud: Cloudinary

Authentication: JWT access token (expires after 3h) and refresh token (expires after 7d)

2.5 Design and Implementation Constraints

Fully developed using Flutter

Initially supports two languages (English and Arabic)

Backend with Node.js and Express

2.6 User Documentation

In-app guide

Onboarding screens during first-time use

2.7 Assumptions and Dependencies

User has internet access

User grants notification permissions

3. External Interface Requirements

3.1 User Interfaces

Login Screen: Sign in using email or Google

Home Screen: Display daily habit list

Add Habit Screen: Create a new habit with recurrence and time settings

Statistics Screen: Display achievement percentage using pie or table view

Reminder Settings: Set daily notification time

3.2 Hardware Interfaces

Not applicable

3.3 Software Interfaces

JWT Authentication System

Backend API (Node.js / Express)

MongoDB

Flutter Notification Packages

3.4 Communication Interfaces

HTTPS

RESTful API

4. System Features

4.1 Add Habit

Inputs: Habit name, time, number of days

Outputs: New habit saved in the database

User Interaction: User clicks the + button, fills in the form, and clicks "Add"

4.2 Daily Tracking

Inputs: User confirms completion

Outputs: Update of achievement status

User Interaction: Interface displays today's habits

4.3 Statistics

Inputs: None (data calculated automatically)

Outputs: Habit achievement chart

User Interaction: User accesses the statistics tab

5. Non-functional Requirements

5.1 Performance Requirements

The app must load within 2 seconds

Each interaction must respond in under 300ms

5.2 Security Requirements

Data encryption using backend rules

Prevent unauthorized access to personal habits

JWT token expires after 3 hours

Refresh token expires after 7 days

5.3 Usability Requirements

Design based on simplicity and clarity

Supports dark mode

5.4 Reliability and Availability

The app should be operational 99% of the time

Supports temporary local data storage when offline

6. Other Requirements

Future ability to share progress with friends

Support for home screen widgets (currently Android only)

7. User Stories

As a user, I want to easily add a daily habit so I can improve my routine without complications.

As a user, I want to receive reminders about my habits so I don't forget to perform them.

As a user, I want to see weekly reports about my progress.

8. Use Case

Use Case: Add New Habit

Actor: User

Precondition: User is logged in

Main Flow:

User clicks "+" button

Fills in habit name and time

Selects frequency

Clicks "Add"

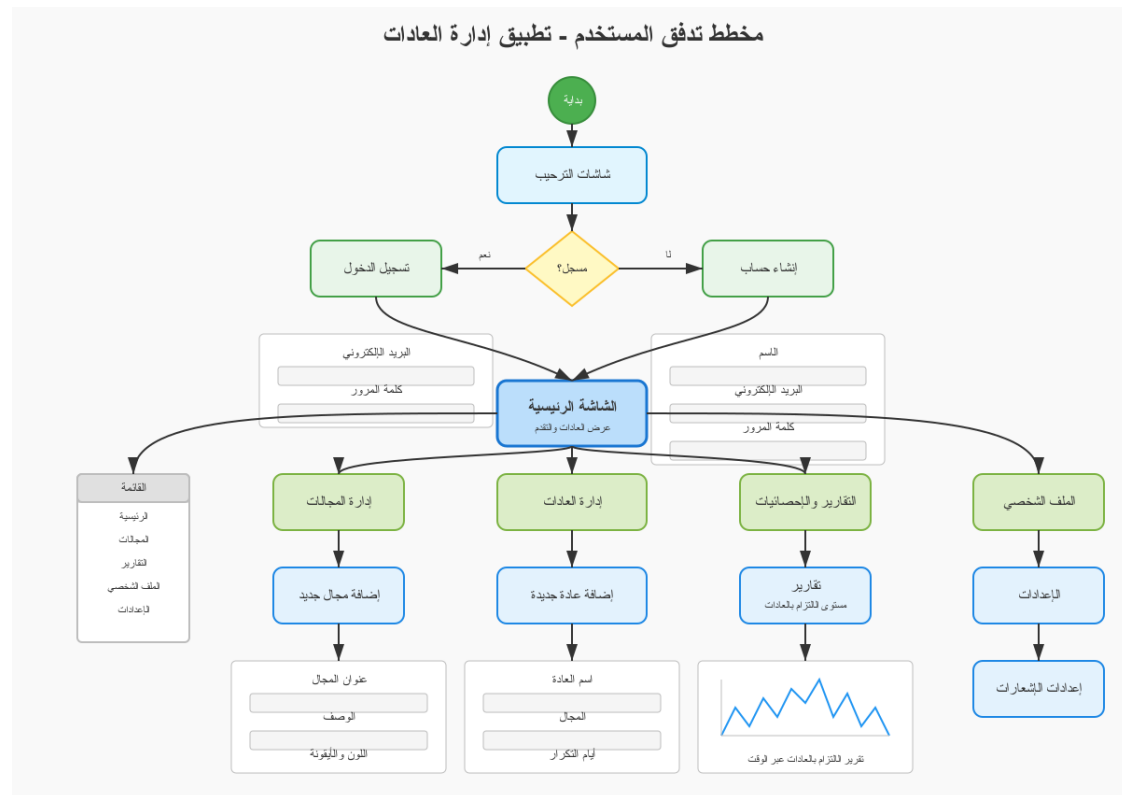
Habit is saved and displayed in the list

Postcondition: New habit appears in today's list

Exception: Missing fields trigger validation message

9. UML & Diagrams

User Flow Diagram



10. ERD – Entity Relationship Diagram

Purpose: To illustrate the database and relationships between entities in the app.

Entities and Attributes:

User

user_id (PK)

name

email

password

Habit

habit_id (PK)

user_id (FK)

title

time

frequency

is_completed

Notification

notification_id (PK)

habit_id (FK)

message

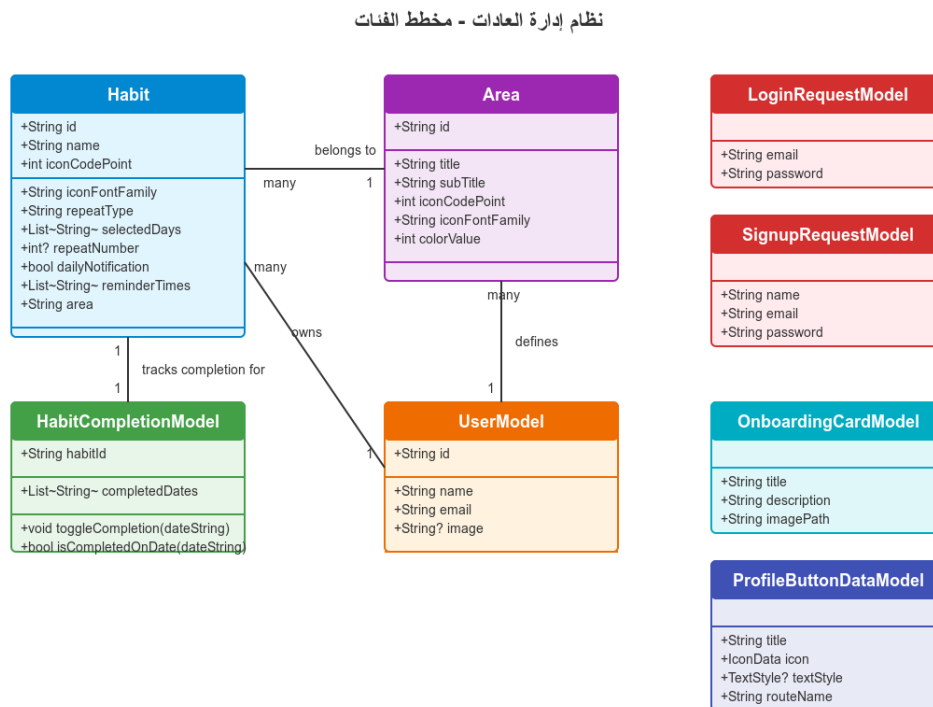
scheduled_time

Relationships:

One User can have many Habits (1:N)

One Habit can have many Notifications (1:N)

Class Diagram:



11. Wireframes – Initial Screen Sketches

Purpose: To visualize the basic layout of the app screens.

Suggested Screens:

Splash Screen

App logo

App name: "Trakify"

Login / Signup Screen

Email and password fields

Login button

Link to create a new account

Home Screen

List of daily habits

Button to add a new habit

Tabs (Today – Weekly – Stats)

Add Habit Screen

Input for habit title

Set time and recurrence

Save button

Statistics Screen

Graph view showing user progress

Count of completed vs. missed habits

Wireframe Drawing Instructions:

Use boxes and icons to represent basic UI elements

Keep the layout clean and simple to guide further programming or detailed design

12. Technologies Used

Flutter for cross-platform development

Cubit for state management

Backend: Node.js / Express

Database: MongoDB, local storage, cache

Cloud: Cloudinary

Auth: JWT and refresh token system (Token expires after 3h, Refresh token after 7d)